

	Maestría en Software	Docente: Ing. Patsy Malena Prieto, MSc.
	PATRONES DE DISEÑO DE APIs	Período Lectivo: Septiembre 2026 Febrero 2027

		FORMATO DE GUÍA DE PRÁCTICA DE LABORATORIO / TALLERES / CENTROS DE SIMULACIÓN – PARA DOCENTES	
MAESTRIA: Software		ASIGNATURA: Patrones de diseño de APIs	
NRO. PRÁCTICA:	06	TÍTULO PRÁCTICA: Despliegue de Aplicaciones Node.js en Producción utilizando AWS, Nginx y PM2	
OBJETIVO Desplegar una Aplicación en un entorno de producción			
INSTRUCCIONES (Detallar las instrucciones que se dará al estudiante):		1. Configurar una infraestructura de red segura en AWS EC2	
		2. Configurar Nginx como un proxy inverso y balanceador de carga	
		3. Gestionar procesos en segundo plano, clusters de procesamiento y variables de entorno seguras con PM2 .	
		4. Automatizar el flujo de Integración y Despliegue Continuo (CI/CD)	
		5. Implementar un sistema automatizado de monitorización de fallos y alertas en tiempo real mediante Webhooks	
ACTIVIDADES POR DESARROLLAR			
Objetivos de la Práctica			
Al finalizar este laboratorio, el estudiante estará en capacidad de:			
1. Configurar una infraestructura de red segura en AWS EC2 para tráfico web (HTTP/HTTPS).			
2. Configurar Nginx como un proxy inverso y balanceador de carga para un backend en Node.js utilizando direcciones IP estáticas.			
3. Gestionar procesos en segundo plano, clusters de procesamiento y variables de entorno seguras con PM2 .			
4. Automatizar el flujo de Integración y Despliegue Continuo (CI/CD) utilizando PM2 Deploy y GitHub .			
5. Implementar un sistema automatizado de monitorización de fallos y alertas en tiempo real mediante Webhooks .			
Requisitos Previos e Infraestructura			
• Una cuenta activa en Amazon Web Services (AWS) .			
• Una instancia EC2 con Ubuntu Server (22.04 LTS o 24.04 LTS).			
• Una cuenta de GitHub con un repositorio (público o privado) de una aplicación Node.js funcional que escuche en el puerto 3000.			
• Una cuenta de Discord o Slack con permisos para crear Webhooks.			
Fase 1: Aislamiento de Red y Configuración del Servidor (AWS)			
Paso 1.1: Configuración del Grupo de Seguridad (Firewall)			
1. Inicia sesión en la consola de AWS y dirígete a EC2 > Instancias .			
2. Selecciona tu instancia, haz clic en la pestaña Seguridad y entra a tu Grupo de Seguridad .			
3. Haz clic en Editar reglas de entrada (Edit inbound rules) y añade las siguientes reglas:			

- **Regla 1:** Tipo: HTTP | Puerto: 80 | Origen: Anywhere-IPv4 (0.0.0.0/0).
- **Regla 2:** Tipo: HTTPS | Puerto: 443 | Origen: Anywhere-IPv4 (0.0.0.0/0).
- **Regla 3:** Tipo: SSH | Puerto: 22 | Origen: Mi IP (Por seguridad).

4. Guarda las reglas. *Nota: El puerto 3000 de Node.js NO debe abrirse al público.*

Nombre	ID de la regla del grupo ...	Intervalo de p...	Protocolo	Origen
-	sgr-003ac155fd68969f7	22	TCP	100.50.83.159/32
-	sgr-02c751a4865b26464	80	TCP	0.0.0.0/0
-	sgr-02688a4bd758c9484	443	TCP	0.0.0.0/0

Paso 1.2: Asignación de IP Elástica (IP Fija)

1. En el menú izquierdo de EC2, navegar a **Red y seguridad > IP elásticas (Elastic IPs)**.
2. Haz clic en **Asignar la dirección IP elástica** y luego en **Asignar**.
3. Seleccionar la IP generada, ve a **Acciones > Asociar la dirección IP elástica**.
4. Seleccionar la Instancia EC2 correspondiente y haz clic en **Asociar**.
5. Copiar la IP pública asignada (ejemplo: 54.123.45.67).

Nombre	Dirección IPv4 asign...	Tipo	ID de asignación
-	100.50.83.159	IP pública	eipalloc-03d8740754ec639ac

Fase 2: Configuración del Servidor Web y Proxy Inverso (Nginx)

Conectarse al servidor AWS vía SSH desde una computadora local (`ssh -i "tu-llave.pem" ubuntu@TU_IP_ELASTICA`) y ejecuta los siguientes comandos. O directamente AWS se puede ingresar a la consola de EC2

2.1. Node.js y NPM (Entorno de Ejecución)

Express es un framework de Node.js, por lo que necesita el motor de ejecución. Es altamente recomendable instalar la versión **LTS (Long Term Support)** utilizando el gestor de versiones NodeSource:

```
# Descargar e instalar el script de configuración para la versión LTS actual
curl -fsSL https://nodesource.com | sudo -E bash -

# Instalar Node.js y NPM
sudo apt-get install -y nodejs
sudo apt install npm -y
```

Para verificar que se instalaron correctamente, ejecuta `node -v` `npm -v`.

2.2. Git (Para transferir el código)

Para clonar el repositorio de la aplicación desde GitHub o GitLab hacia el servidor.

```
sudo apt install git -y
```

2.3. Herramientas de Compilación (Build Essentials)

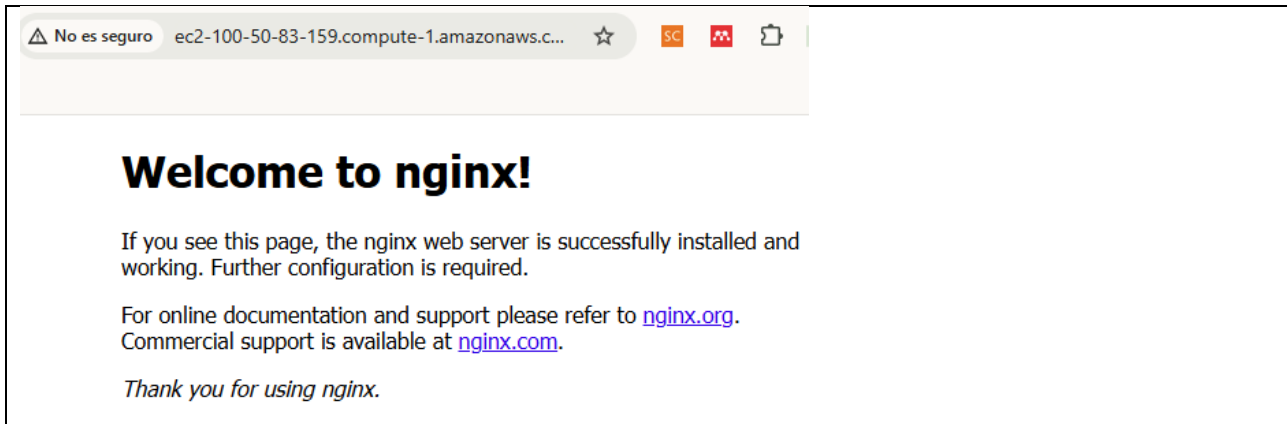
Muchas librerías de Node.js (como bcrypt, crypto o conectores de bases de datos) compilan código en C o C++ durante el npm install. Si no tienes estas herramientas, la instalación de dependencias fallará.

```
sudo apt install build-essential -y
```

2.4: Instalación de Nginx

```
sudo apt update && sudo apt upgrade
sudo apt install nginx -y
# Permitir tráfico HTTP (puerto 80) y HTTPS (puerto 443)
sudo ufw allow 'Nginx Full'
# Comprobar el funcionamiento de Nginx
sudo systemctl status nginx
```

Si abres su navegador web e ingresas la **dirección IP de tu servidor** (o `http://localhost` si está en el entorno local), debería ver la página de bienvenida por defecto de Nginx.



2.5: Configuración de la IP como sitio por defecto

1. Abrir el archivo de configuración por defecto:

```
sudo nano /etc/nginx/sites-available/default
```

2. Borrar el contenido y pegar el bloque del proxy inverso. Esta configuración intercepta las peticiones externas del puerto 80 y las redirige internamente al puerto 3000 de la app:

```
server {  
    listen 80 default_server;  
    listen [::]:80 default_server;  
  
    server_name _;  
  
    location / {  
        proxy_pass http://localhost:3000;  
        proxy_http_version 1.1;  
        proxy_set_header Upgrade $http_upgrade;  
        proxy_set_header Connection 'upgrade';  
        proxy_set_header Host $host;  
        proxy_cache_bypass $http_upgrade;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
}
```

3. Verifica la sintaxis del archivo y reinicia el servicio:

```
sudo nginx -t  
sudo systemctl reload nginx
```

Comandos útiles de gestión

- **Detener Nginx:** sudo systemctl stop nginx
- **Iniciar Nginx:** sudo systemctl start nginx
- **Recargar configuración (sin desconectar usuarios):** sudo systemctl reload nginx

Fase 3: Clonar el Proyecto de Express desde GITHUB a AWS EC2 (Elastic Compute Cloud)

3.1: Vincular AWS EC2 con GitHub

1. Regresa a la terminal del servidor **AWS EC2**. Genera un par de llaves SSH:

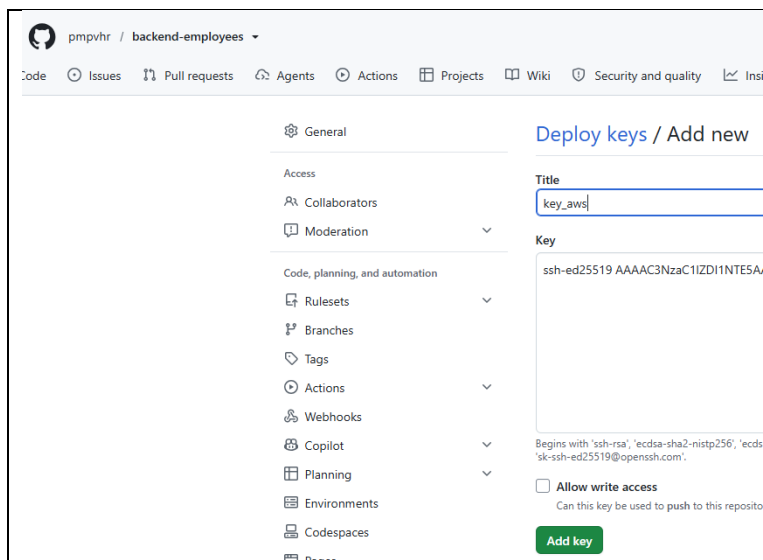
```
ssh-keygen -t ed25519 -C "servidor-produccion"  
# Presiona ENTER consecutivamente para omitir contraseñas
```

2. Imprime y copia la llave pública:

```
cat ~/.ssh/id_ed25519.pub
```

```
ubuntu@ip-172-31-32-85:~$ ssh-keygen -t ed25519 -C "servidor-produccion"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/ubuntu/.ssh/id_ed25519):
Enter passphrase for "/home/ubuntu/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/ubuntu/.ssh/id_ed25519
Your public key has been saved in /home/ubuntu/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:vPok+UNTUJ5RPr+lGrc0DKt3e6kFhTi6bVYMFjEaWWk servidor-produccion
The key's randomart image is:
+--[ED25519 256]--+
|      o=**      |
|      .ooE+...  |
|      o+=..... |
|      . + ++. + |
|      So  o+B   |
|      .o.o..=.o |
|      o.o.o.+o o..|
|      =. o. ..o. |
|      ..o.  .o.  |
|      +-----[SHA256]-----+
```

- Dirígete a tu repositorio en **GitHub**, ve a **Settings > Deploy keys > Add deploy key**. Pega la clave pública y asígnale un nombre descriptivo. (No necesita marcar los permisos de escritura).



- Valida la conexión ejecutando en AWS:

```
ssh -T git@github.com
# Escribe 'yes' cuando te pregunte si confías en el host.
```

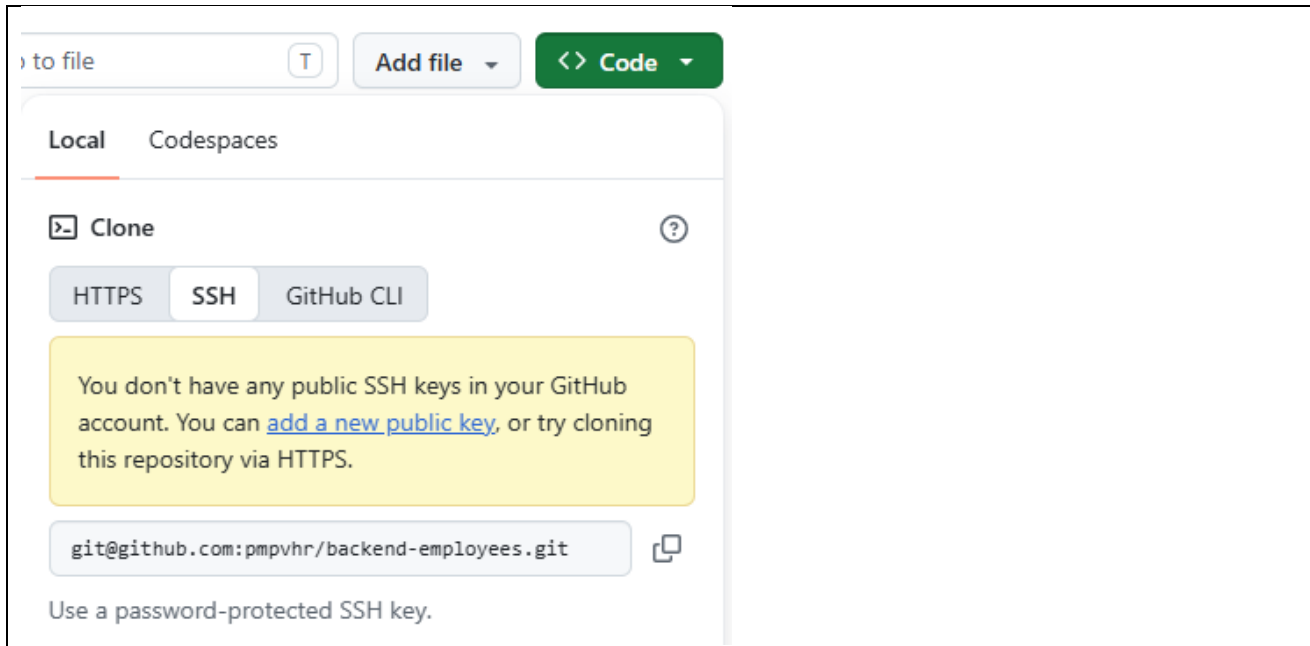
3.2: Asignación de Permisos de Directorio (AWS)

Crea la carpeta de destino del despliegue y otorga la propiedad del directorio al usuario operativo de Ubuntu:

```
sudo mkdir -p /var/www/tu-app
sudo chown -R ubuntu:ubuntu /var/www/tu-app
```

3.3: Clonar el repositorio de GITHUB

1. Copiar el código SSH que se encuentra en GITHUB



2. En la terminal de AWS ejecutar el commando

```
git clone URL_DE_GITHUB_QUE_COPIASTE
```

3. Instalar todos los módulos asociados a este proyecto

```
npm i express morgan mongoose cors  
npm install tsx  
npm install typescript@latest ts-node@latest ts-node-dev@latest -D  
npm install typescript @types/node @types/express --save-dev
```

Fase 4: Gestión de Procesos y Variables de Entorno (PM2- Process Manager 2)

4.1: Instalación Global de PM2

Permanecer en la terminal de AWS e instalar PM2 de manera global en el sistema:

```
sudo npm install pm2 -g
```

Paso 4.2: Estructuración del archivo Ecosistema (Local)

Abre una **nueva terminal en la computadora local** (dentro de la carpeta del proyecto Node.js).

1. Inicializa el ecosistema de PM2:

```
npm install pm2 -g  
pm2 init
```

2. Modificar el archivo generado ecosystem.config.js incorporando las variables de entorno de producción y los parámetros de automatización de Git:

```
module.exports = {
  apps : [{
    name: "mi-node-app",
    script: "./index.js",
    instances: "max",    // Modo Cluster: usa todos los núcleos de la CPU
    exec_mode: "cluster",

    // Producción: Variables de entorno protegidas
    env: {
      NODE_ENV: "production",
      PORT: 3000,
      DB_HOST: "://amazonaws.com",
      DB_USER: "db_admin",
      DB_PASS: "password_seguro_de_base_de_datos"
    },
    // Logs y Monitoreo del Servidor
    error_file: "/var/www/tu-app/logs/err.log",
    out_file: "/var/www/tu-app/logs/out.log",
    log_date_format: "YYYY-MM-DD HH:mm:ss Z",
    merge_logs: true
  }],

  // Automatización del Despliegue desde tu PC local
  deploy : {
    production : {
      user : 'ubuntu',
      host : 'TU_IP_ELASTICA_AQUÍ',
      ref : 'origin/main',
      repo : 'git@github.com:TU_USUARIO/TU_REPOSITORIO.git',
      path : '/var/www/tu-app',
      'post-deploy' : 'mkdir -p logs && npm install && pm2 reload ecosystem.config.js --env production
&& pm2 save',
      ssh_options: "IdentityFile=~/.ssh/tu-llave-aws.pem" // Ruta a tu llave .pem local
    }
  }
};
```

- Guarda los cambios, añade el archivo .config al archivo .gitignore si tus credenciales son críticas, y sube tus cambios a GitHub:

```
git add .
git commit -m "feat: agregar configuracion de pm2"
git push origin main
```

Una vez instaladas estas 5 herramientas, los pasos lógicos para ponerla en marcha son:

1. Clonar el código con git clone.
2. Instalar dependencias con npm install.
3. Iniciar la app Express con PM2: `pm2 start app.js --name "mi-express-app"`.
4. Configurar el archivo `/etc/nginx/sites-available/default` apuntando el `proxy_pass` a `http://localhost:3000`.

Fase 5: Autenticación Segura y Despliegue Automatizado (CI/CD)

5.1: Ejecución del Despliegue Inicial (Desde la Computadora Local)

Regresa a la terminal de **tu computadora local** y ejecuta la inicialización de carpetas de PM2:

```
# 1. Preparar la estructura en AWS
pm2 deploy production setup
# 2. Desplegar el código, instalar dependencias y levantar el proceso
pm2 deploy production
```

Fase 6: Monitorización Activa, Gestión de Logs y Alertas

6.1 Creación de la cuenta y uso de Incoming Webhooks en Slack

1. Entra a **Slack App Directory** e inicia sesión.
2. Busca la aplicación llamada Incoming WebHooks y haz clic en Añadir a Slack.
3. Selecciona el canal de texto (ej. `#alertas-servidor`) donde quieres recibir los mensajes de PM2.
4. Haz clic en Añadir integración con Incoming WebHooks.
5. Copia la URL que aparece en el campo URL de Webhook (comienza con `https://hooks.slack.com/...`).

Publicar en el canal

Emplea por elegir el canal donde publicará mensajes tu Webhook entrante.

#todo-maestría

[o crear un nuevo canal](#)

Agregar integración con WebHooks entrantes

Al crear una webhook entrante, aceptas los [Términos de servicio de la API de Slack](#).

Instrucciones de configuración

Te iremos indicando los pasos necesarios para que configures un webhook entrante y empieces a enviar datos a Slack.

[cerrar](#)

URL de webhook

<https://hooks.slack.com/services/T0C40TR4H9R/B0C42LZLCBK/1tXBpP32DnmvxjKexE16uUJ>

Envío de mensajes

Tienes dos opciones para enviar datos a la URL del webhook anterior:

- Enviar una secuencia JSON como parámetro de la **carga útil** en una solicitud POST
- Enviar una secuencia JSON como cuerpo de una solicitud POST

En un mensaje sencillo, tu carga útil JSON puede contener una propiedad de **texto** como mínimo. Este es el texto que se publicará en el canal.

Un ejemplo sencillo:

```
payload={"text": "Esta es una línea de texto en un canal.\nY esta es otra línea de texto."}
```

Esto se mostrará en el canal como:

6.2: Configuración de Notificaciones Automáticas (AWS)

Vuelve a ingresar a la consola de **AWS** e instala el módulo de alertas integrándolo con un Webhook de Discord o Slack:

```
# Instalar el módulo de notificaciones global
sudo npm install pm2-notify -g

# Configurar el canal (Elige Discord o Slack según corresponda)
pm2 set pm2-notify:discordUrl "URL_DE_TU_WEBHOOK_AQUÍ"
# pm2 set pm2-notify:slackUrl "URL_DE_TU_WEBHOOK_AQUÍ"

# Filtrar para recibir únicamente eventos de crasheos (caídas) o apagados
pm2 set pm2-notify:events "error,exit"
pm2 set pm2-notify:apps "mi-node-app-Alumno"
pm2 save --force

#Para visualizar el log desde consola
pm2 logs pm2-notify

#Para listar los procesos
pm2 list

#Parar e iniciar un proceso
pm2 stop mi-node-app
pm2 start mi-node-app
```

6.3: Instalación de Rotación de Logs (Prevención de Disco Lleno)

```
pm2 install pm2-logrotate
```

Pruebas de Verificación y Criterios de Evaluación

Para validar el éxito del laboratorio, realice las siguientes pruebas de calidad de infraestructura:

1. **Prueba de Red:** Accede desde el navegador de tu computadora a `http://TU_IP_ELASTICA`. Debe ver la interfaz de tu aplicación Node.js sin necesidad de especificar el puerto `:3000`.
2. **Prueba de Resiliencia (Simulación de caída):** Conéctase a AWS y apaga la app manualmente con `pm2 stop mi-node-app`. Compruebe que llegue una alerta inmediata al canal de Discord/Slack. Vuelve a encenderla con `pm2 start mi-node-app`.
3. **Prueba de CI/CD:** Realiza una pequeña modificación visual en tu código local, haga un git push a GitHub, y ejecuta desde tu PC `pm2 deploy production`. Refrescar el navegador y comprueba que los cambios se hayan aplicado sin haber tocado la consola de AWS.

CONCLUSIONES:

- Optimización del Rendimiento y Concurrencia mediante la Separación de Roles: La implementación de Nginx como proxy inverso demostró ser una solución arquitectónica eficiente para entornos de producción. Al delegar la recepción del tráfico público (puerto 80) a Nginx, se libera a la aplicación de Node.js de tareas de bajo nivel (como la gestión de conexiones TCP y cabeceras de red), permitiéndole concentrarse exclusivamente en la ejecución de la lógica de negocio y optimizando el uso de recursos del servidor.
- Alta Disponibilidad y Resiliencia con el Modo Clúster de PM2: Node.js está diseñado inherentemente sobre un solo hilo de ejecución. Mediante el uso del Modo Cluster de PM2 (instances: "max"), se logró multiplicar el rendimiento de la aplicación al instanciar automáticamente réplicas del backend en todos los núcleos de vCPU disponibles en la instancia de AWS EC2. Esto garantiza un balanceo de carga interno y la autorecuperación del servicio ante excepciones no controladas sin interrumpir la experiencia del usuario.
- Gobernanza de Seguridad y Reducción de la Superficie de Ataque: A través de la correcta configuración de los Grupos de Seguridad de AWS, se validó el principio de mínimo privilegio en redes. Al abrir únicamente los puertos web estándar (80/443) y de administración (22 para SSH), y mantener el puerto interno de la aplicación (3000) completamente aislado del tráfico exterior, se mitigan vectores de ataque críticos y escaneos de puertos maliciosos directamente hacia el backend.
- Eficiencia Operativa mediante la Automatización de Despliegues (CI/CD): La integración de PM2 Deploy con GitHub eliminó la necesidad de realizar procesos manuales propensos a errores humanos, como conectarse por SSH para hacer git pull o reiniciar servicios manualmente. Este flujo de trabajo automatizado permite actualizar el entorno de producción en un solo comando desde la máquina local, reduciendo drásticamente el Time-to-Market y garantizando despliegues con cero tiempo de inactividad (Zero-Downtime Deployment).
- Monitoreo Proactivo y Prevención de Degradación del Servidor: El acoplamiento de alertas automatizadas a través de Webhooks (Discord/Slack) transforma la gestión de infraestructura de un modelo reactivo a uno proactivo, notificando fallos críticos en tiempo real. Asimismo, la incorporación del módulo de rotación de logs mitigó el riesgo técnico latente de denegación de servicio por almacenamiento lleno, asegurando la sostenibilidad y estabilidad operativa de la infraestructura de AWS a largo plazo.

RECOMENDACIONES:

- **Migración Obligatoria a HTTPS (Cifrado de Datos):** Aunque el laboratorio se ejecutó utilizando una dirección IP estática sobre HTTP (puerto 80), para entornos productivos reales es mandatorio **adquirir un nombre de dominio** y configurar un certificado **SSL/TLS gratuito utilizando Certbot (Let's Encrypt)**. Esto garantiza el cifrado de datos de extremo a extremo y evita que los navegadores marquen el sitio como "No Seguro".
- **Externalización y Centralización de las Bases de Datos:** Se recomienda firmemente **no instalar bases de datos (como PostgreSQL, MySQL o MongoDB) dentro de la misma instancia EC2** donde corre la aplicación de Node.js. En su lugar, se deben utilizar servicios administrados como **AWS RDS** (para bases de datos relacionales) o **AWS DynamoDB**. Esto desacopla el almacenamiento de la lógica de cómputo, facilitando respaldos automáticos y previniendo caídas totales si la instancia EC2 se satura.

	Maestría en Software	Docente: Ing. Patsy Malena Prieto., MSc.
	PATRONES DE DISEÑO DE APIs	Período Lectivo: Septiembre 2026 Febrero 2027

Docente / Técnico Docente:

Firma: _____